

Lekcja

Temat: Powtórzenie z list, krotek, słowników, zbiorów.

Rozwiązywanie zadań

1. Mutowalność (mutable) vs niemutowalność (immutable)

Typ	Mutowalność	Czy można zmienić po utworzeniu?
list	mutowalna	✓ można dodawać, usuwać, zmieniać elementy
dict	mutowalny	✓ można dodawać, usuwać, zmieniać pary klucz-wartość
set	mutowalny	✓ można dodawać, usuwać elementy
tuple	niemutowalna	✗ NIE można zmienić! Jest jak "zamrożona lista"

1. List (lista) – MUTOWALNA

```
my_list = [1, 2, 3]
```

```
my_list[0] = 100      # zmiana elementu
my_list.append(4)      # dodanie elementu
my_list.pop(0)         # usuwa 1 element
my_list.remove(3)      # Usunięcie elementu po wartości
del my_list[0]         # Usunięcie przez del
my_list.clear()        # Wyczyszczenie całej listy
```

```
print(my_list)
```

2. Dict (słownik) – MUTOWALNY

```
my_dict = {"a": 1, "b": 2}
```

```
my_dict["a"] = 100    # zmiana wartości
my_dict["c"] = 3       # dodanie nowego klucza
my_dict.pop("a", None) # Usunięcie elementu (bezpieczne – jak discard)
my_dict.pop("b")       # usunięcie po kluczu
del my_dict["c"]       # Usunięcie przez del
```

```
my_dict.clear()          # Wyczyszczenie słownika
```

```
print(my_dict)
```

3. Set (zbiór) – MUTOWALNY

```
my_set = {1, 2, 3}
```

```
my_set.add(4)
my_set.remove(2)          # usunięcie z błędem
my_set.discard(3)         # usunięcie bez błędu
my_set.pop()              # usuwa losowy element
my_set.clear()            # wyczyszczenie
```

```
print(my_set)
```

4. Tuple (krotka) – NIEMUTOWALNA

```
my_tuple = (1, 2, 3)
```

```
# my_tuple[0] = 10 # BŁĄD TypeError: 'tuple' object does not support item
assignment
```

```
del my_tuple              # Możesz usunąć tylko całą krotkę:
my_tuple.append(4)         # brak takiej metody
my_tuple.clear()          # Nie istnieje clear() dla tuple.
```

Ważny przypadek (tuple z listą w środku)

```
my_tuple = (1, [2, 3])
```

```
my_tuple[1].append(4)
```

```
print(my_tuple)           # (1, [2, 3, 4])
```

Krotka jest niemutowalna, ale **obiekt w środku (lista)** już nie.

2. Podstawowe metody dla każdego typu

LISTA (mutowalna)

```
lista = [1, 2, 3]
```

```
# Dodawanie
```

```
lista.append(4)           # [1, 2, 3, 4] - dodaje na koniec
```

```

lista.insert(1, 99)    # [1, 99, 2, 3, 4] - wstawia na pozycję
lista.extend([5, 6])  # [1, 99, 2, 3, 4, 5, 6] - łączy listy

# Usuwanie
lista.remove(99)       # [1, 2, 3, 4, 5, 6] - usuwa pierwsze wystąpienie
element = lista.pop()  # usuwa i zwraca ostatni (6)
lista.pop(0)           # usuwa i zwraca element z indeksu 0 (1)
lista.clear()          # [] - usuwa wszystko

# Inne przydatne
lista = [3, 1, 4, 1, 5]
lista.sort()           # [1, 1, 3, 4, 5] - sortuje
lista.reverse()        # [5, 4, 3, 1, 1] - odwraca
len(lista)             # 5 - długość
3 in lista             # True - sprawdza czy istnieje

```

KROTKA (niemutowalna) - mało metod

```

krotka = (1, 2, 3, 2, 4)

# Nie można dodawać ani usuwać!
# Ale można odczytywać:
print(krotka[0])      # 1
print(krotka.count(2)) # 2 - ile razy występuje
print(krotka.index(3)) # 2 - na którym indeksie jest 3
print(len(krotka))    # 5

# Jedyne sposoby "dodania" - stworzenie nowej
nowa = krotka + (5, 6) # (1, 2, 3, 2, 4, 5, 6)

```

SŁOWNIK (mutowalny)

```

sloownik = {"a": 1, "b": 2}

# Dodawanie / zmiana
sloownik["c"] = 3      # {"a":1, "b":2, "c":3}
sloownik.update({"d": 4}) # dodaje wiele

# Usuwanie
del sloownik["b"]       # {"a":1, "c":3, "d":4}
sloownik.pop("c")       # usuwa i zwraca wartosc 3 dla klucza c
sloownik.popitem()      # usuwa i zwraca ostatnią parę ('d', 4)
sloownik.clear()        # {}

# Dostęp

```

```
print(slownik.get("a", 0)) # 1 (bez błędu jeśli nie ma)
print(slownik.keys()) # widok kluczy
print(slownik.values()) # widok wartości
print(slownik.items()) # widok par (klucz, wartość)
```

ZBIÓR (mutowalny) - elementy unikalne, nieuporządkowane

```
zbior = {1, 2, 3}

# Dodawanie
zbior.add(4) # {1, 2, 3, 4}
zbior.add(2) # {1, 2, 3, 4} - nic się nie dzieje (już jest)

# Usuwanie
zbior.remove(3) # {1, 2, 4} - błąd jeśli nie ma
zbior.discard(10) # {1, 2, 4} - brak błędu jeśli nie ma
zbior.pop() # usuwa i zwraca jakiś element (nie wiadomo który)
zbior.clear() # {}

# Operacje na zbiorach
A = {1, 2, 3}
B = {3, 4, 5}
print(A | B) # {1,2,3,4,5} - suma
print(A & B) # {3} - przecięcie
print(A - B) # {1,2} - różnica
```